SECURITE DES SYSTEMES DISTRIBUES

Incident Desk

Authentification et autorisation
Spring Security, JWT et React

Rapport technique - version du 11 novembre 2026

Une application de declaration et de suivi des incidents informatiques, avec des droits differents pour les employes, techniciens et administrateurs.

Perimetre	Resultat
Backend	Spring Boot 3.4.2, Spring Security, JPA, H2, JJWT 0.13.0
Frontend	React, Vite, React Router, Axios
Verification executee	40 controles HTTP reussis + verification de l'interface dans le navigateur
Documents	README, diagrammes, collection Postman, guide de demonstration, captures de l'application
Evidence a completer	Les neuf captures dans l'interface native Postman restent a prendre.

Objectif pedagogique

Distinguer l'authentification (identifier un utilisateur), l'autorisation par role (autoriser une action) et le controle des donnees (verifier le proprietaire ou le technicien affecte). Une route React masquee ne suffit pas : toute requete est controlee par le backend.

Le projet reprend les notions de l'atelier TP2. Le code, les tests et ce rapport doivent etre compris et presentes par l'etudiant. Aucun resultat Postman ni aucune capture de cet outil ne sont inventes.

1. Architecture generale

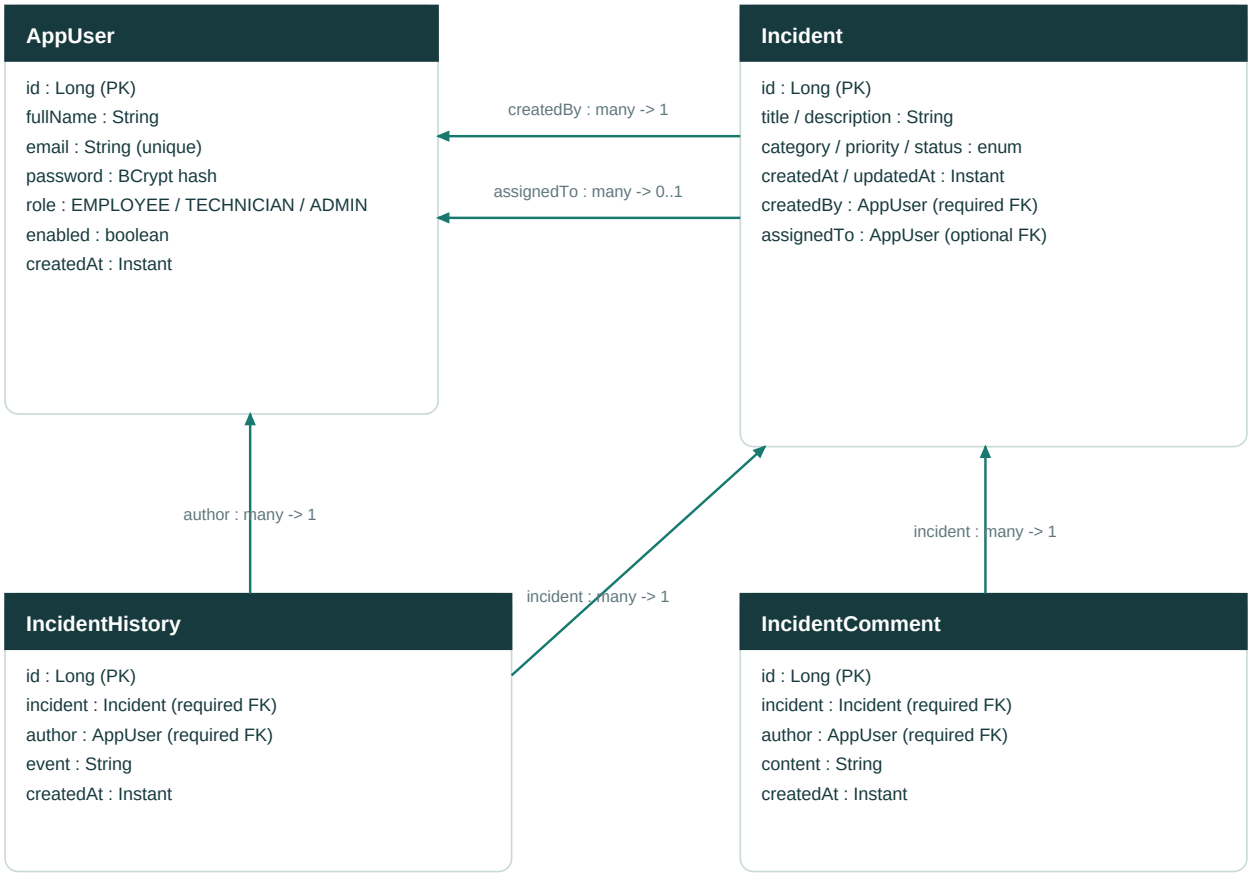


React envoie le JWT dans l'entete Authorization. Le filtre valide sa signature et son expiration, charge le compte et construit l'authentification. Les controles de role et de propriete precedent toute lecture ou modification sensible.

Le backend utilise SessionCreationPolicy.STATELESS : il ne depend pas d'une session HTTP pour authentifier les requetes. La base conserve les comptes, incidents, notes et evenements. CORS autorise localhost:5173 et 127.0.0.1:5173.

Les DTO evitent d'exposer le hash du mot de passe. La validation utilise @NotBlank, @Email, @Size, @NotNull et @Valid. Un @RestControllerAdvice traduit les erreurs applicatives en JSON ; les erreurs du filtre de securite ont des gestionnaires 401/403 dedies.

2. Modele de donnees



Comment.author also references AppUser (many -> 1). All links use @ManyToOne LAZY.

Un utilisateur peut creer plusieurs incidents ; chaque incident possede exactement un createur. L'affectation est optionnelle, mais doit viser un technicien actif. Chaque note et chaque evenement possedent un incident et un auteur.

Choix JPA : @ManyToOne represente le cote portant la cle etrangere. FetchType.LAZY differe le chargement de l'objet associe ; EAGER impose son chargement. Une collection @OneToMany inverse serait possible, mais les listes sont obtenues ici par des requetes de repository : cela evite des collections bidirectionnelles inutiles. Les DTO sont construits dans une transaction et les getters permettent l'initialisation des proxies Hibernate.

Les entites utilisent Instant pour les dates. Les historiques tracent creation, affectation, changement de statut et ajout de note. CLOSED est terminal : toute tentative de modification ulterieure est refusee.

3. Authentification et autorisations

Role	Droits
EMPLOYEE	Creer un incident, consulter ses propres incidents et modifier son profil.
TECHNICIAN	Consulter ses incidents affectes, changer leur statut, ajouter une note.
ADMIN	Consulter tous les incidents/utilisateurs, affecter, activer/desactiver, consulter les statistiques.

Inscription : le role EMPLOYEE est fixe par le serveur. Le DTO ne contient pas de role ; une tentative d'injection de ADMIN ne donne aucun privilege. Le mot de passe est encode avec BCrypt et compare via matches lors de la connexion.

Claims JWT : sub identifie l'email ; iat represente la creation ; exp limite la validite a cinq minutes ; role indique le role. JJWT ajoute ces champs avec subject, issuedAt, expiration et claim, puis signe le token. Le parseur verifie la signature et rejette un token expire. Un JWT signe n'est pas un chiffrement.

Role : @EnableMethodSecurity active @PreAuthorize. hasRole('ADMIN') correspond a l'autorite ROLE_ADMIN ; hasAuthority compare une autorite exacte. Le filtre relit le role et enabled en base : un compte desactive ne peut plus utiliser son ancien JWT.

Propriete : Authentication.getName() fournit l'email authentifie. IncidentService charge l'utilisateur et compare son identifiant a createdBy ou assignedTo. Meme un appel direct a /api/incidents/{id} ne contourne pas ce controle.

Profil : le nom et l'email sont modifiables. Un nouveau JWT est emis apres modification. Le changement de mot de passe exige l'ancien mot de passe correct et une confirmation identique.

Refresh token : extension facultative non implementee. Un access token autorise l'API pour une courte duree ; un refresh token sert a obtenir un nouvel access token et demande une politique de rotation/revocation. Ici, l'expiration exige une nouvelle connexion.

4. Tests de securite executes

Cas obligatoire	Attendu	Observe
01 No JWT	401	401
02 Invalid JWT	401	401
03 Expired correctly signed JWT	401	401
04 Employee admin route	403	403
05 Other employee incident	403	403
06 Unassigned technician update	403	403
07 Disabled login	401	401
08 Duplicate email	409	409
09 Invalid incident data	400	400

40/40 controles HTTP reussis. Le fichier docs/test-results.json conserve les reponses. Le script tests/verify.py est reproductible et ajoute des donnees de demonstration sans effacer les anciennes.

Le cas du JWT expire utilise un token correctement signe avec la cle de demonstration et une date exp dans le passe. Il teste donc l'expiration, et non simplement une signature invalide.

Controles supplementaires : JWT d'un utilisateur desactive, employe tentant une affectation, refus de modifier CLOSED, incident inexistant, inscription avec role ADMIN, confirmation/ancien mot de passe, mise a jour du profil et nouveau token.

Statut	Interpretation
400	Donnees manquantes, invalides ou incoherentes.
401	Authentification absente, invalide, expiree ou compte desactive.
403	Utilisateur authentifie mais sans autorisation.
404	Ressource introuvable.
409	Email deja utilise ou conflit avec l'etat CLOSED.

Postman : la collection est fournie et automatise les tokens, les identifiants et les assertions. Son execution native et les neuf captures sont a finaliser selon docs/POSTMAN-CAPTURES.md. Les resultats ci-dessus proviennent du script HTTP, pas de captures Postman.

5. Scenario fonctionnel et interface

Le scenario HTTP cree deux incidents pour le premier employe et un pour le second. Il verifie l'isolation, affecte un incident via ADMIN, passe de ASSIGNED a IN_PROGRESS puis RESOLVED, ajoute une note et verifie que le proprietaire retrouve le nouvel etat et l'historique.

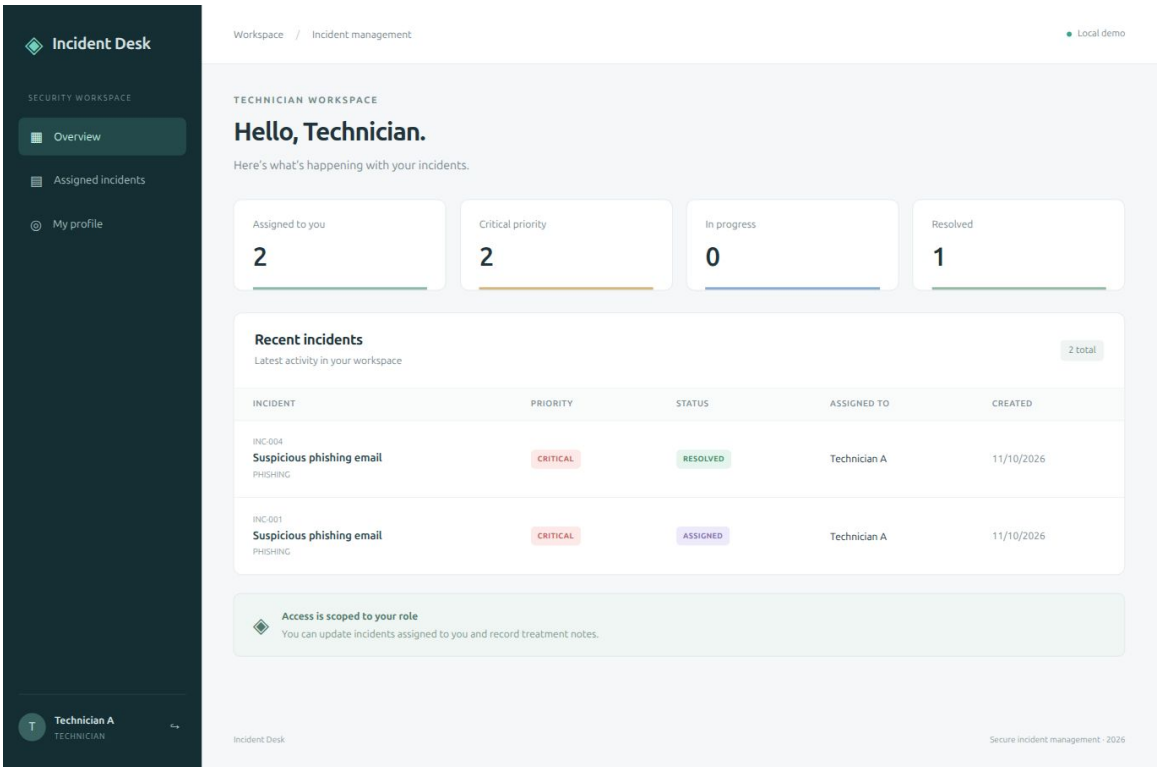


Figure 1. Dashboard du technicien : incidents affectes et compteurs adaptes au role.

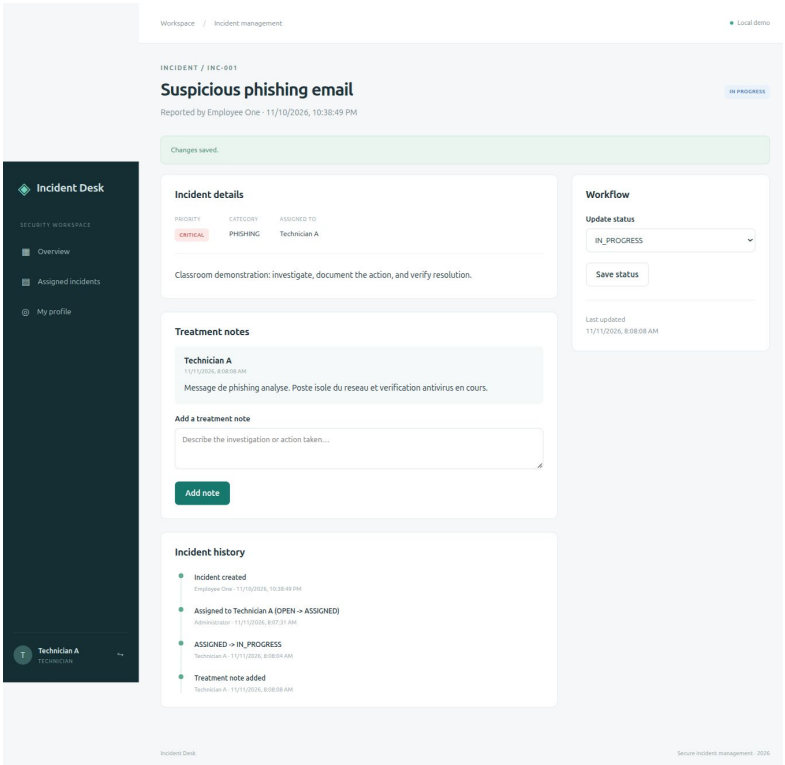


Figure 2. Traitement dans React : changement de statut, note et historique sauvegardes.

6. Captures complementaires

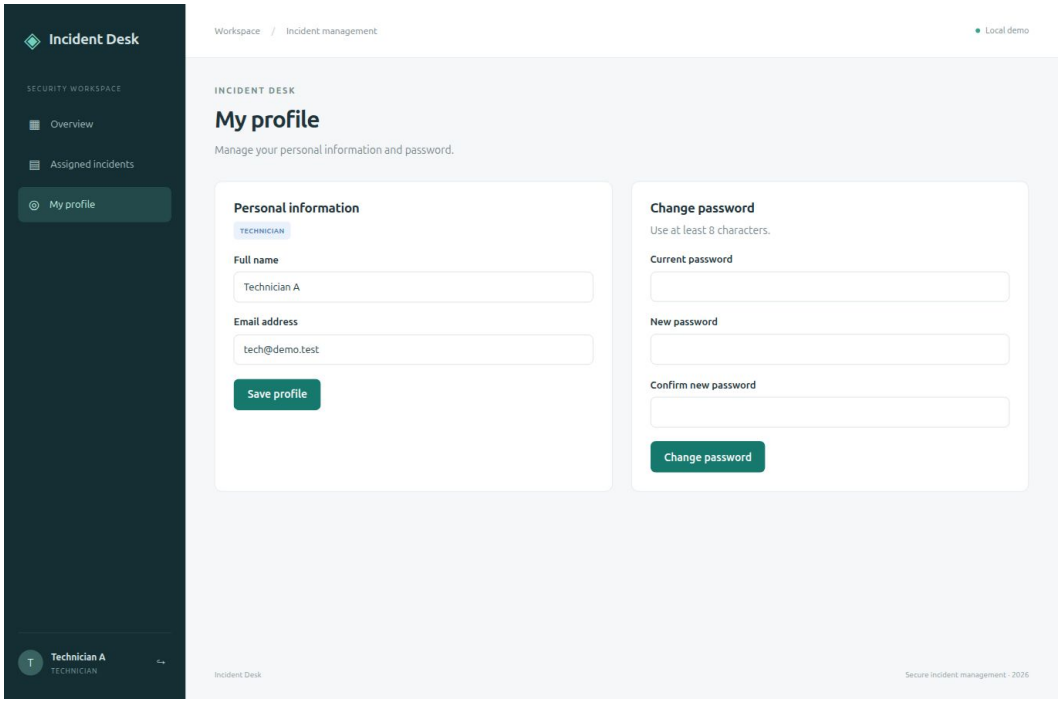
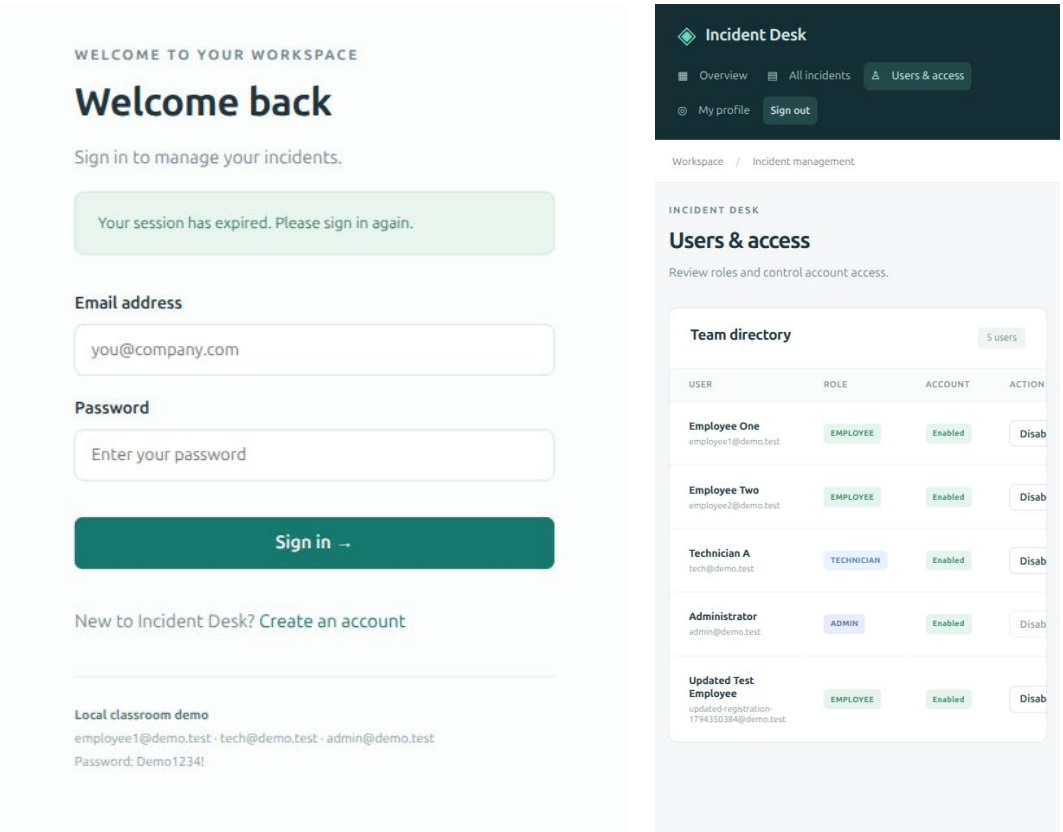


Figure 3. Profil et formulaire de changement de mot de passe.

Les autres captures reelles sont livrees dans docs/screenshots : expiration de session, dashboard employe, historique, dashboard administrateur, gestion des utilisateurs et affectation.



Figures 4 et 5. Message de session expiree ; gestion des utilisateurs dans une fenetre etroite.

Les controles visuels ont couvert les mises en page etroite et bureau, les connexions des trois roles, l'affectation, la modification de statut, l'ajout de note et la redirection d'un technicien tentant une route ADMIN.

7. Execution, demonstration et limites

Lancement : bash start-backend.sh puis bash start-frontend.sh, dans deux terminaux. Ouvrir http://127.0.0.1:5173. Le backend utilise 8080. Les quatre comptes initiaux utilisent Demo1234! : employee1@demo.test, employee2@demo.test, tech@demo.test et admin@demo.test.

Demonstration : employe cree un incident ; administrateur l'affecte ; technicien ajoute une note et le resout ; employe consulte l'historique. Montrer ensuite les refus 401 et 403 dans Postman. Le guide docs/DEMO-5-MINUTES.md detaille cette sequence.

Limites explicites : cle JWT et comptes connus pour la demonstration, console H2 publique selon le sujet, aucun refresh token. Un changement de mot de passe ne revoque pas les JWT deja emis ; ils restent limites par leur expiration. Un compte desactive est bloque immediatement. La deconnexion efface sessionStorage. La session navigateur reste exposee a une eventuelle faille XSS : cette application est une demonstration locale, pas un deploiement de production.

Livrables : projets backend et frontend ; README ; present rapport ; diagrammes SVG et sources Mermaid ; captures de l'application ; collection Postman et resultats HTTP ; guide de demonstration. Les captures natives Postman et la demonstration devant le professeur restent a effectuer.

References techniques

Spring Security - method security

JJWT - claims, signature et parsing

Axios - interceptors

Jakarta Persistence - ManyToOne

Sujet de reference : devoir_jwt_spring_security_react.pdf, sections 1-27. Le code et les observations locales sont la source des resultats annonces.